BRNO UNIVERSITY OF TECHNOLOGY — FACULTY OF INFORMATION TECHNOLOGY

http://fit.vut.cz

Project practice 2025/2026

Evaluating Test Completeness Against Software Requirements

Bc. Jakub Bláha

Abstract
What is the problem? What is the topic? The aim of this paper? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. How is the problem solved, the aim achieved (methodology)? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. Curabitur massa neque, commodo posuere fringilla ut, cursus at dui. Nulla quis purus a justo pellentesque. What are the specific results? How well is the problem solved? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. So what? How useful is this to Science and to the reader? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod.

Keywords: Keyword1 — Keyword2 — Keyword3

Supplementary Material: Demonstation video — Downloadable code

*herout@fit.vut.cz, *Faculty of Information Technology, Brno University of Technology*

1. Introduction

With the rise of Large Language Models (LLMs) and the increase in their capabilities, companies are motivated to use LLMs for automation tasks, including testing and the tooling for it. However, LLMs are not plug-and-play; specialized tooling is often required to produce reliable outputs. This work defines a formal base into which LLMs rewrite requirements and test cases written in natural language, so that the result is parsable by test coverage checking systems.

There is an increasing demand to instrument systems in natural language, including the specification of requirements and their test cases for testing systems and system components. Even when requirements and test cases are written in English, they must still be rewritten into a mathematical formal base so that existing test-coverage techniques can be applied. A formal base into which LLMs can convert human-written text is therefore needed. This formal base must be able to express all motivational requirement examples. Its semantics must be easily interpretable by LLMs and agentic systems, so that agents can rewrite requirements and test case descriptions from human language into the formal base with high accuracy and without changing the semantics; the formalism itself must be unambiguous. It must also be convertible to existing formal bases used by formal verification tools. A good formalism should meet all of the mentioned criteria.

2. Existing methods of requirement formalisation – State of the Art

The translation from natural-language (NL) requirements to formal specifications has long been pursued through controlled-language frontends that constrain the input enough to make it unambiguous and further processable. The recent maturation of large language models has reopened the question of to what degree this translation can be automated without such restrictions, leading to LLM-based pipelines and hy-

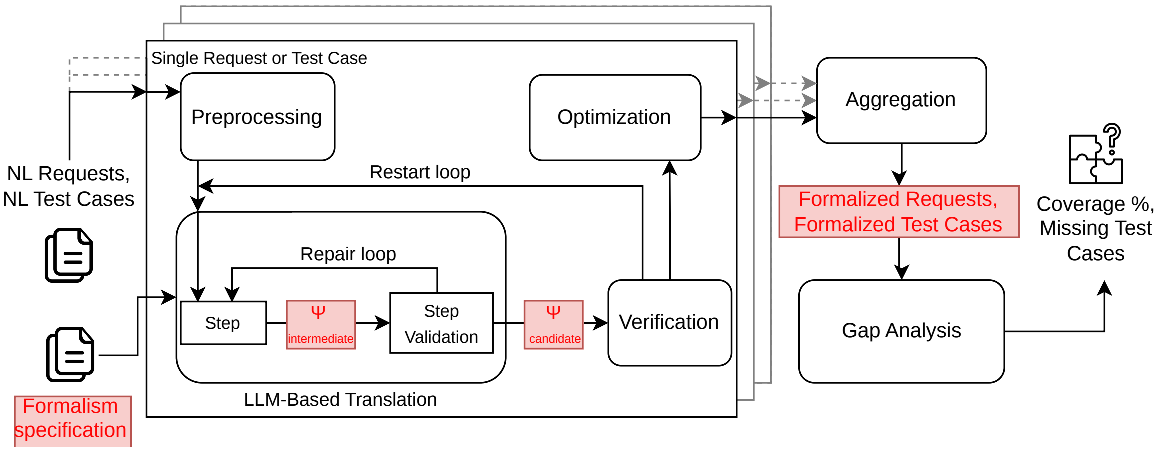

Figure 1. Preliminary design of the verification pipeline. The components highlighted in red are the contribution of this work: the *Formalism specification* that defines the target formal language, the intermediate and candidate representations Ψ (a requirement or test case expressed using the formalism), and the *Formalized Requests / Formalized Test Cases* that result from the translation. The focus of this work is therefore on defining the formalism itself and its formal properties, not on the LLM-based translation pipeline.

brid approaches that pair LLMs with formal provers and solvers — the rest of this section surveys that landscape. This overview is informed by the recent short survey of Beg et al. (2025) [1], which classifies the LLM-based literature in detail. It is further complemented with controlled-language frontends that the survey only briefly covers. Table 1 summarises the systems discussed in this section.

Controlled-language frontends. The longest-lived line of work attacks the problem at the *input* side by constraining the natural language itself. Nowakowski et al. (2013) [9] developed the *Requirements Specification Language* and the *ReDSeeDS* platform, which couples a constrained-NL requirements notation with automated transformations into code. Ghosh et al. (2016) [5] present *ARSENAL*, an NLP-based framework that extracts formal requirement specifications from natural language with automatic verification. NASA's *FRET*, introduced by Giannakopoulou et al. (2020) [6], lets engineers write requirements in *FRETISH*, a restricted English with unambiguous semantics that maps deterministically to temporal logic.

LLM-only pipelines. With the rise of large language models, a growing body of work delegates the full translation to the LLM itself. Nayak et al. (2022) [8] present *Req2Spec*, an NLP pipeline that maps natural-language requirements onto the pattern-based formal specifications consumed by HANFOR; on 222 BOSCH automotive requirements it correctly formalises 71%. Cosler et al. (2023) [2] propose *nl2spec*, which uses few-shot prompting of Codex (`code-davinci-002`) and Bloom-176B to translate unstructured natural language into Linear-time Temporal Logic (LTL), with iterative user refinement of subformulas. Fang et al. (2024) [3] introduce *AssertLLM*, a three-stage pipeline of customised GPT-4o models (the final stage retrieval-augmented) that generates SystemVerilog Assertions from hardware design specifications with 89% correctness.

Hybrid LLM + prover systems. The most recent strand pairs an LLM with a formal prover or solver that has the final say on correctness. Jiang et al. (2022) [7] introduce *Thor*, which couples a 700M-parameter decoder-only transformer with the Isabelle/HOL theorem prover via Sledgehammer-based premise selection ("Hammers"), raising proof success on PISA from 39% to 57%. Yang et al. (2023) [11] release *LeanDojo* and the retrieval-augmented prover *ReProver*, a ByT5 encoder-decoder paired with premise retrieval over the Lean mathematics library and benchmarked on 98 734 theorems. Quan et al. (2024) [10] propose *Explanation-Refiner*, a refinement loop between an LLM (GPT-4, GPT-3.5-turbo, Llama2-70b, Mixtral-8x7b, or Mistral-small) and the Isabelle/HOL prover that validates and corrects natural-language explanations. Fazelnia et al. (2024) [4] build *SAT-LLM*, which combines an LLM with an SMT solver to detect conflicting requirements at F1 0.91.

The systems surveyed above translate natural-language requirements into a single, fixed target formalism – LTL for nl2spec, SystemVerilog Assertions for AssertLLM, HANFOR patterns for Req2Spec, prover-specific languages for Thor, LeanDojo and Explanation-Refiner – and each commits up front to one back-end logic. Retargeting the same requirement to a different

Table 1. State-of-the-art systems for natural-language to formal-specification translation discussed in this section, in chronological order within each category. Abbreviations: HITL = human-in-the-loop; NLP = natural language processing; MDD = model-driven development; SAL = Symbolic Analysis Laboratory; LTL = linear-time temporal logic; HANFOR = an industrial requirements-analysis tool that consumes pattern-based formal specifications; SVA = SystemVerilog Assertions; HOL = higher-order logic (as in the Isabelle/HOL theorem prover); SMT = satisfiability modulo theories.

Ref.	System	Year	Models	Target	HITL
Controlled-language frontends					
[9]	RSL/ReDSeeDS	2013	— (rule-based)	Code (MDD)	Yes
[5]	ARSENAL	2016	NLP pipeline	SAL/LTL	No
[6]	FRET	2020	— (rule-based)	LTL	Yes
LLM-only pipelines					
[8]	Req2Spec	2022	NLP pipeline	HANFOR	No
[2]	nl2spec	2023	Codex Bloom-176B	LTL	Yes
[3]	AssertLLM	2024	GPT-4o	SVA	No
Hybrid LLM + prover systems					
[7]	Thor	2022	700M transformer	Isabelle/HOL	No
[11]	LeanDojo	2023	ByT5 + retrieval	Lean	No
[10]	Explanation-Refiner	2024	GPT-4/3.5 Llama2-70b Mixtral-8x7b Mistral-small	Isabelle/HOL	No
[4]	SAT-LLM	2024	LLM + SMT solver	SMT	No

verifier therefore means rewriting it.

The formalism presented in the next section sits as an intermediate layer between natural language and the back-end verifiers: close enough to English that a human or an LLM can read and produce it, but precisely defined mathematically so that every construct has unambiguous semantics. It is logic-agnostic, so the same requirement can be projected to several back-end verifiers without rewriting it. It is also positioned as a shared representation for requirements and the matching test cases – the same predicates appear on both sides, which is what makes coverage-gap detection mechanical. The construct names (*Causes*, *Happening*, *MaxVal*, *EvtOccCount*, . . .) are drawn from the vocabulary humans already use when describing requirements informally, and every name is tied directly to a precise mathematical definition. A close precedent is nl2spec's interactive subformula refinement, which similarly attaches English fragments to formal atoms; but nl2spec's mapping is user-defined per requirement – the engineer labels their own subformulas during each translation – whereas here the vocabulary is a fixed, shared catalogue that the LLM and the reader can rely on across every requirement.

3. Contributing by designing a "close-to-NL" formalism

While prior controlled-language frontends [6, 9, 5] and LLM-based translators [2] target a single fixed logic

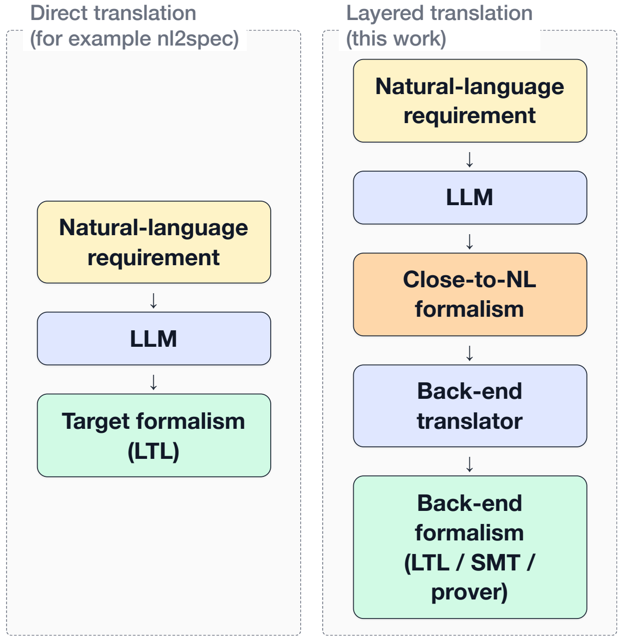

Figure 2. Direct translation (left) targets a single back-end formalism in one step. The proposed layered translation (right) inserts a close-to-NL formalism between the LLM and the back-end, so the same intermediate representation can be projected to several different verifier languages by a deterministic back-end translator.

— typically LTL or a tool-specific pattern language — none of the systems surveyed in Section 2 position their formalism around predicate-level matchability between requirements and test cases for coverage-gap detection, nor as a logic-agnostic intermediate explicitly designed to be the translation target for LLMs. The formalism presented in this section is designed around precisely these two properties.

3.1 Structure of the formalism

The formalism is organised into several layers, each building on the ones below it. The remainder of this section refers back to these layers when discussing individual requirements and design choices.

Time entities, and traces. The bottom layer fixes a time set T (either discrete or continuous, for a given requirement) and a set of *entities* representing the kinds of things present in the system: signals, storage (registers), communication channels, states, events, and so on. A *trace* is a function that assigns to each time point a partial *valuation* mapping entities to their current values.

Expressions and point-in-time predicates. Built on top of traces, *expressions* compute values from a trace (for example "the value of register A at time t" or "the number of times e_A has occurred up to t"), and *predicates* are boolean-valued expressions (for example "A is greater than 5"). Both kinds implicitly take the trace and an ambient time argument that is supplied by the surrounding context rather than passed explicitly.

Operations. Operations (*write*, *fire*, *transmit*, *set_state*, ...) are the actions a test case fires to drive a trace. Each operation, applied to an entity at a specific time, produces an effect predicate describing what must hold in the trace as a result of the firing.

Temporal constructs. Temporal constructs (*Always*, *Eventually*, *Initial*, *Causes*, *CausesWithin*, *Sequence*, ...) lift point-in-time predicates into *trace predicates*: a single boolean statement about the whole trace, obtained by quantifying over time. Trace predicates compose with one another via the usual logical operators ($\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$) at the trace level, and together they form the building blocks of a requirement's constraint.

Requirements and test cases. A *requirement* is a triple consisting of a time flavour, a set of participating entities, and a trace predicate (the constraint). A *test case* is a triple consisting of an initial valuation, a finite set of stimuli (timed operation firings), and a set of point-in-time assertions to verify at specific times. Both sides reference the same entities and draw their predicates from the same space, which is what allows a checker to determine which sub-formulas of a requirement are exercised by the test cases.

3.2 Formalism requirements and its design

The formalism satisfies the properties listed below. Each property is paired with a concrete use case expressed in abstract placeholders (signals A, B; events e_A, e_B; etc.) and a brief note on the construct the formalism provides to address it.

Running example. To anchor the discussion, consider the following requirement – Req 10 from the worked-example set in Appendix A.

The software shall enter the emergency mode if any of the following failures individually occurs twice in less than 10 seconds: (a) sensor failure, (b) energy failure, (c) communication failure.

The requirement is formalised over the following entities:

Entity	t_e	μ_e
sensor_failure	EventTrigger	—
ev_sensor_fail	Event	$\{target \mapsto id_{sensor\_failure}, type \mapsto generic\}$
energy_failure	EventTrigger	—
ev_energy_fail	Event	$\{target \mapsto id_{energy\_failure}, type \mapsto generic\}$
comm_failure	EventTrigger	—
ev_comm_fail	Event	$\{target \mapsto id_{comm\_failure}, type \mapsto generic\}$
emergency_mode	State	—

With time set to the continuous flavour (units: seconds), the constraint is:

$$
\begin{aligned}
Causes\Big(&\Big(Happening(ev\_sensor\_fail, Now) \\
&\qquad \wedge EvtOccCount_\rho(\\
&\qquad\quad ev\_sensor\_fail,\\
&\qquad\quad MkIntervalOO(Now - 10, Now)\\
&\qquad) \geq 1\Big)\\
&\vee \Big(Happening(ev\_energy\_fail, Now)\\
&\qquad \wedge EvtOccCount_\rho(\\
&\qquad\quad ev\_energy\_fail,\\
&\qquad\quad MkIntervalOO(Now - 10, Now)\\
&\qquad) \geq 1\Big)\\
&\vee \Big(Happening(ev\_comm\_fail, Now)\\
&\qquad \wedge EvtOccCount_\rho(\\
&\qquad\quad ev\_comm\_fail,\\
&\qquad\quad MkIntervalOO(Now - 10, Now)\\
&\qquad) \geq 1\Big),\\
&Val_\rho(emergency\_mode, Now) = true \Big)
\end{aligned}
$$

The constructs *Causes*, *Happening*, *EvtOccCount*, and *MkIntervalOO* are defined in Appendix A; for the present discussion only the overall shape matters. Each disjunct in the antecedent says "one of the failure types just fired now *and* fired at least once within the last ten seconds", and the consequent records that the system has entered the emergency mode.

Letting ψ_R denote the constraint above and E_R the entity set listed in the table, the requirement is formally the triple

$$R = (\text{Continuous}, E_R, \psi_R).$$

A matching test case fires two sensor failures within ten seconds and asserts that the emergency mode is active at the time of the second failure:

$$
\begin{aligned}
TC_R &= (\sigma_0, S, A), \quad \text{where}\\
\sigma_0 &= \{emergency\_mode \mapsto false\},\\
S &= \{(1, fire, sensor\_failure),\\
&\qquad (5, fire, sensor\_failure)\},\\
A(5) &= \{Val_\rho(emergency\_mode, 5) = true\}.
\end{aligned}
$$

Meta-level properties. **Mechanical coverage-gap detection.** Given a set of formalised requirements and a set of formalised test cases, the formalism shall enable a checker to determine which requirement conditions are not exercised by any test case. *Solution:* Requirements and test cases are formalised over the same entities and the same predicates, so that the same sub-formulas appear on both sides. A checker can then evaluate, for each test case, which meaningful combinations of sub-formulas inside a requirement's formula are driven to true and which to false, and from that compute the fraction of relevant combinations that the test suite exercises.

Logic-agnostic representation of system behaviour in time. The formalism shall not commit to a specific underlying logic, so that a single requirement representation can be retargeted to different verification back-ends. *Solution:* A requirement is represented as a composition of two kinds of building blocks: (i) ordinary expressions and predicates that, when evaluated against a trace (the same kind of object a test case produces), yield concrete values or truth values at each point in time; and (ii) temporal predicates that aggregate those point-in-time predicates into a single truth value over the entire trace. The semantics of each building block is defined directly in set-theoretic terms over traces and time, independent of any target back-end. Every temporal predicate has a direct LTL (or metric-LTL) equivalent and a corresponding timed-automaton construction.

Extensible domains. Real-world requirements draw on an open-ended range of kinds of things in a system (signals, registers, channels, states, and so on) and of events that can happen to them (a register being written, a channel transmitting a value, a state being entered), and it is unrealistic to fix a closed catalogue up front that covers every domain the formalism may ever be applied to. The formalism shall therefore keep these catalogues extensible. *Example:* To support a new entity kind C representing network packets, only the relevant catalogue is widened; the rest of the formalism is untouched. *Solution:* The set of entity kinds, the set of value types, the assignment of permitted attributes to each entity kind, and the assignment of permitted events (parameterised by the entity they happen to) to each entity kind are all parameters of the formalism rather than fixed sets. Adding a new entry to any of them requires only registering it in the relevant catalogue; existing constructs continue to work unchanged.

Time and values. **Discrete and continuous time.** The formalism shall support both discrete-time and continuous-time requirements within a single vocabulary. *Example:* A discrete-time requirement says "at every step, A is incremented by 3"; a continuous-time requirement says "e_B must occur within 2 seconds of e_A". *Solution:* The time set T has a discrete flavour T_D and a continuous flavour T_C; each requirement carries a *flavour* $\in \{\text{Discrete}, \text{Continuous}\}$ field that fixes T for its scope.

Entities of multiple kinds carry values, and operations modify them in test cases. The formalism shall represent entities of different kinds (registers, states, communication channels, computed signals, etc.) and let each carry a value at any given time point. It shall also provide a way for test cases to describe how those values change over time. *Example:* Register A, state B, and channel C each have a value at every relevant time point; a test case may, for instance, write a new value into A, transition B to a new state, or transmit a value through C. *Solution:* The formalism defines an extensible catalogue of entity kinds (currently signal, storage, event, event-trigger, channel, state, value, abstract, and set), and a valuation maps each entity to its value at each time point of a trace. Modifications of those values are expressed through *operations* — an extensible catalogue of actions, currently including *calculate*, *write*, *read*, *fire*, *set_state*, *transmit*, *receive*, *insert*, *remove*, and *clear* — that a test case fires at specific time points to drive the trace. Each entity kind has its own subset of applicable operations (e.g. *write* applies to storage, *fire* to event-triggers, *transmit* to channels).

Static entity-attribute access. The formalism shall expose static attributes of an entity (independent of trace and time). *Example:* The address of register A is the constant `0xAA000018` and can be referenced as such. *Solution:* Static attributes are stored in an entity's modifier map $\mu_e : \mathit{ModKey} \rightharpoonup \mathit{ModVal}$; accessors such as $\mathit{Addr}(e) = \mu_e(\mathit{address})$ retrieve them without reference to ρ or t.

Reference to an entity's value at another time point. The formalism shall allow a constraint to refer to the value an entity had (or will have) at an earlier or later time point, enabling recurrences, toggles, and look-ahead conditions. *Example:* "$A(\mathit{Now}) = A(\mathit{Now} - 1) + 3$"; "$B$ equals the negation of B's previous value"; or "C at the next step equals the current value of D". *Solution:* $\mathit{Val}_\rho(e,t)$ returns the value of e at any time t, past or future; for discrete time, *Prev*, *Next*, and *RelTime* shift the time argument backward or forward; $\mathit{ValBefore}_\rho(e,t)$ exposes the left limit at t.

Arithmetic on values. The formalism shall support standard arithmetic on numeric values so that constraints can express computations over entity values. *Example:* "$A = B + 3$"; or "$C = 1.2 \cdot D$". *Solution:* The standard arithmetic operators $+$, $-$, $\times$, $/$ are defined on numeric values and can appear inside any expression.

Time arithmetic and intervals. The formalism shall support arithmetic on time points and the explicit construction of intervals with controlled endpoint openness. *Example:* "the duration elapsed between e_A and e_B"; "the half-open interval $(t_1, t_2]$". *Solution:* *Diff* returns the duration between two time points; the four interval constructors *MkIntervalCC*, *MkIntervalOC*, *MkIntervalCO*, *MkIntervalOO* build intervals with the chosen open/closed endpoints.

Events and triggers. **Event occurrence at a given time point.** The formalism shall express that an event occurs at a particular time point. *Example:* "e_A is happening at t." *Solution:* The predicate $\mathit{Happening}(e, t)$, where e is an entity of type `Event`.

Past-event semantics. The formalism shall express that an event has occurred at some point up to the current time. *Example:* "e_A has happened at least once." *Solution:* The predicate $\mathit{HasHappened}_\rho(e, t)$.

Reference to event-occurrence times. The formalism shall expose the time of a specific past or future occurrence of an event. *Example:* "the time of the most recent occurrence of e_A." *Solution:* $\mathit{LastOcc}_\rho(ev, t, n)$ returns the interval spanning the n most recent occurrences of ev up to t; $\mathit{FirstOcc}_\rho$ does the same in the forward direction.

Counting event occurrences within an interval. The formalism shall count how many times an event occurred within a given interval, with control over whether interval endpoints are included. *Example:* "the number of occurrences of e_A in the half-open interval $(t_1, t_2]$." *Solution:* $\mathit{EvtOccCount}_\rho(ev, i)$ returns the count.

Conditional and logical structure. **Conjunction, disjunction, and negation of predicates.** The formalism shall combine predicates of any kind (event happenings, value comparisons, occurrence counts, historical values) into a single conjunctive or disjunctive guard, and shall negate any predicate. *Example:* "e_A and $B > 100$ and e_C has occurred at least twice"; "e_A or e_B or e_C"; or "A is not equal to B". *Solution:* *AllOf*, *AnyOf*, and *Not* form the conjunction, disjunction, and negation of predicates drawn from the shared predicate space.

Value-conditional branches. The formalism shall support constraints that branch on the current value of an entity. *Example:* "If A is true then B equals X; otherwise B equals Y." *Solution:* A value-guarded branch is expressed as the conjunction of two implications using *AllOf*, *Implies*, and the comparison predicate *Cmp*, for example: $\mathit{AllOf}\big(\{\mathit{Implies}(\mathit{Cmp}(A, =, \mathit{true}), \mathit{Cmp}(B, =, X)), \mathit{Implies}(\mathit{Cmp}(A, =, \mathit{false}), \mathit{Cmp}(B, =, Y))\}\big)$.

Sets, set membership, and set quantification. The formalism shall represent finite sets of values, test whether a value belongs to a set, quantify over a finite set of entities, and reason about properties of the subset that satisfies a predicate. *Example:* "the value v belongs to set S"; "for every entity in $\{A, B, C, D\}$, predicate P holds"; or "at most two entities in $\{A, B, C, D\}$ have a value greater than 10." *Solution:* The Set entity kind carries set values; *Contains* tests membership; the quantifiers $\mathit{ForAll}(S, P)$ and $\mathit{Exists}(S, P)$ range over a finite set, with *Filter* and *Size* supporting cardinality reasoning over the subset satisfying a predicate.

Temporal patterns. **Initial-value constraints.** The formalism shall be able to constrain the value of an entity at the initial time point. *Example:* "Signal A has initial value 0." *Solution:* The temporal construct $\mathit{Initial}(\psi) \Leftrightarrow \psi(\rho, 0)$ asserts a point-in-time predicate at time 0.

Existence, global constraints, and trace-level composition. The formalism shall express both "at some point in the trace" and "at every time point in the trace", and shall let multiple such trace-level constraints be combined into a single requirement. *Example:* "Eventually A equals 5"; "A is always non-negative"; or "Always ψ_1 *and* eventually ψ_2". *Solution:* $\mathit{Eventually}(\psi)$ and $\mathit{Always}(\psi)$ produce trace-level truth values; multiple trace predicates compose via the standard logical operators ($\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$) applied at the trace level.

Sequencing and time-bounded causation. The formalism shall require multiple actions to occur in a specified order, and shall require an effect to occur within a bounded duration of a cause. *Example:* "e_A then e_B then e_C"; "if e_A occurs, e_B must occur within 2 seconds". *Solution:* $\mathit{Sequence}(\psi_1, \ldots, \psi_n)$ and $\mathit{CausesWithin}(\psi_{cond}, \psi_{effect}, d)$, respectively; similarly, $\mathit{Causes}(\psi_{cond}, \psi_{effect})$ for unbounded causation, $\mathit{Precedes}(\psi_1, \psi_2)$ for ordering, $\mathit{Excludes}(\psi_1, \psi_2)$ for mutual exclusion, and $\mathit{Immediately}(\psi_1, \psi_2)$ for next-step succession.

Sliding temporal windows. The formalism shall reason about a window of fixed length ending at the current time. *Example:* "e_A occurred at least twice in the last 10 seconds." *Solution:* An interval constructor applied to *Now* (e.g. $\mathit{MkIntervalOO}(\mathit{Now} - d, \mathit{Now})$) produces the window, which can then be passed to $EvtOccCount_\rho$ to count event occurrences, to $MaxVal_\rho$/$MinVal_\rho$ to bound a signal's range, or to interval predicates such as *IntervalIncludes*/*IntervalExcludes*.

3.3 Formalism definition

The complete formalism – primitives, entity types, expressions, predicates, operations, temporal constructs, and the definition of a requirement and a test case – is given in Appendix A. The remainder of this section refers to those definitions but does not reproduce them.

3.4 Known limitations and possible future extensions

Word order versus English syntax. The formalism deliberately uses English words for its constructs, expressions, and predicates – *Always*, *Causes*, *MaxVal*, *Happening*, and so on – to keep individual identifiers recognisable to a human reader. The overall syntactic structure, however, is prefix function application throughout: every temporal construct, operation, and predicate places its head before its arguments. A requirement that reads in English as "after receiving a MastershipInfo of BACKUP, the system shall operate as Backup" becomes a nested $Causes(\ldots,\ldots)$ expression whose outer construct binds the cause-effect relation, with the trigger condition as the first argument and the consequence as the second. The mapping from English to the formalism is therefore not a near-monotonic substitution of words but a structural reorganisation. Human readers must mentally re-order the formalism back to natural English to verify it, and an LLM-based translator has to perform an explicit syntactic transformation rather than a token-level rewrite – which is harder to learn, harder to evaluate, and prone to silent re-ordering errors that change the meaning.

Modifier parameterisation. Modifier values are currently static – fixed at entity-definition time to a single element of the modifier value set. Some entities, however, have intrinsic time-dependent behaviour that would naturally be expressed as a function rather than a single value. Consider a free-running cycle counter, modelled as a `Storage` entity, whose value at any time is the time elapsed since the last system reset. The present formalism has no way to express this intrinsic behaviour. A future extension that allowed modifier values themselves to be functions – for example, a modifier whose value is the expression returning the elapsed time since the last reset – would let such entities declare their intrinsic behaviour at definition time.

Value transformations. The formalism has no notion of a transformation applied to an entity's value before that value is used in an expression – for instance, reinterpreting a raw stored bit pattern as a signed integer, or applying a unit conversion or scaling factor. A requirement may need to compare the *signed* interpretation of a register's value rather than its raw stored value; e.g. "the signed value of A is greater than the signed value of A at the previous step". The present formalism exposes only the raw value $Val_\rho(e,t)$ and provides no construct to denote such a reinterpretation. Supporting this would require a catalogue of value-transformation functions, each mapping a value to its transformed value and applicable wherever an expression is expected, analogous to how the arithmetic operators are defined on numeric values.

Partiality propagation. Several functions in the formalism are explicitly partial; representative cases include *Prev* – undefined at $t = 0$; *RelTime* – undefined when the shifted time point would be negative; *MkInterval* – undefined when the start exceeds the end; $ValBefore_\rho$ – undefined when no prior operation has fired; and Val_ρ – undefined when the entity has no value at the given time point. The formalism does not yet specify how partiality propagates when an undefined sub-expression appears inside a larger expression or predicate. The options are: a *well-formedness condition* that forbids such expressions syntactically, requiring the user to guard them explicitly; a *three-valued / Kleene logic* under which undefinedness propagates upward and yields a third truth value; or a *definedness guard* under which any predicate containing an undefined sub-expression evaluates to *false*.

Open question – Channel/transmit semantics. The current model is asynchronous: the transmitted value persists on the channel until the next *transmit*, so *receive* at any $t' \geq t$ reads the last transmitted value, and *transmit* and *receive* are independent operations that may occur at different times. It is unclear whether this is the right model for all channel types, or whether some channels should require the receiver to observe the value at the exact transmission time. An alternative synchronous model would have *transmit* directly cause *received* to fire at the same time t, making *receive* a consequence of *transmit* rather than a separate operation.

Predicate snapshot comparison. The formalism cannot currently express that the set of predicates holding at one time point must match the set of predicates holding at another. Consider a system with four hardware registers R1, R2, R3 and R4, together with a declared pool of predicates over them: for instance, R1 lies between 0 and 100, R2 is less than R3, and R3 is greater than 50. A requirement might state that after the system is reset and a short stabilisation period has elapsed, exactly the same predicates from this pool must hold as held immediately before the reset. The temporal constructs in this formalism can quantify over time and check individual predicates, but they cannot capture which predicates from a given pool hold at a time point and then compare those captured sets across two time points by equality, by subset, or by non-empty intersection. Supporting this would require a snapshot primitive that returns the subset of a declared predicate pool holding at a time point, together with set-level operators on the resulting snapshots.

Time-when expression. The formalism provides $LastOcc_\rho$ and $FirstOcc_\rho$ for asking when a given event last or first occurred, but only for entities of type `Event` – whose value at every time point is a boolean occurrence flag. A general-purpose $LastTrue_\rho(\psi,t)$ that returns the most recent $t' \leq t$ at which an arbitrary predicate ψ holds, together with a symmetric $FirstTrue_\rho(\psi,t)$, would let requirements reference the time of conditions that are not themselves declared events. For instance, the time at which the compound predicate $Val_\rho(R_1,t) > 10 \wedge Val_\rho(R_2,t) < 50$ last held cannot be expressed concisely with the current operators: the conjunction is not a declared event, and no existing construct returns the time at which an arbitrary predicate was last true. Like the predicate snapshot comparison above, this extension requires treating predicates as first-class objects rather than as constraints quantified over by temporal constructs; lifting predicates in this way is not a trivial change to the underlying semantics, which is why both are deferred to future work rather than included in the present formalism.

Time element for continuous requirements. A requirement could optionally declare a *time element* $\varepsilon \in \mathbb{R}_{>0}$, representing the smallest meaningful time increment in that requirement (e.g. a sampling period or clock cycle). In the continuous-time flavour, *Prev* and *Next* are currently undefined because there is no canonical predecessor or successor of a real-valued time point. With a declared ε, they could be lifted to continuous time as $Prev(t) = t - \varepsilon$ and $Next(t) = t + \varepsilon$, making the full set of discrete-time expressions available to requirements that have a known sampling granularity.

Unified time model via time element. The current formalism maintains two separate time flavours, $T_D = \mathbb{N}$ and $T_C = \mathbb{R}_{\geq 0}$, selected per requirement. These could be unified into a single model by making the time element ε (see the time element extension) a mandatory requirement parameter. The unified time set would then be $T = \{0, \varepsilon, 2\varepsilon, 3\varepsilon, \ldots\}$, restricting time to multiples of ε and making $Prev(t) = t - \varepsilon$ and $Next(t) = t + \varepsilon$ total (except at $t = 0$). Under this model, discrete time is simply the special case $\varepsilon = 1$, and a "continuous" requirement with a known sampling period ε is expressed the same way. Crucially, this is a stronger commitment than merely defining *Prev* and *Next* on an otherwise uncountable $\mathbb{R}_{\geq 0}$: it makes the time domain itself countable, so that quantifiers such as $\forall t \in T$ range over a countable set.

Typed sets. The current `Set` entity type stores an arbitrary finite subset of V_{nonset}, with no constraint that all elements of a given set share a uniform type. A single `Set` entity could therefore mix booleans with real numbers or entity identifiers, even though such heterogeneous collections rarely correspond to anything a requirement intends. A future extension would let a `Set` entity declare an element type at definition time – for example, a set of real-valued sensor readings, a set of entity identifiers for the currently active failure conditions, or a set of boolean flags. The declared type would propagate to the operations: $insert(t,e,v)$ and $remove(t,e,v)$ would require v to match the declared type, and aggregation expressions such as *MaxVal* over a set of reals would gain well-defined semantics. This removes a class of silent type mismatches that the coverage checker cannot otherwise detect, and brings the `Set` entity into line with the typed-payload story under *Event payload* below.

Flat sets. Set values in the formalism are flat: a `Set` entity cannot contain another set. This was chosen for simplicity; supporting sets that may themselves contain sets should be considered in a future revision.

Event payload. Currently an event entity carries only a boolean occurrence flag indicating whether the event has fired at a given time point. Any data associated with the event must therefore be expressed as a separate value check on the target entity. For instance, stating that a transmission carried the value TRUE on a particular communication channel requires two distinct assertions: one that the transmit event fired, and another that the channel held the value TRUE at that moment. This splits what logically belongs together into two clauses and risks the value constraint being omitted from the requirement specification, silently leaving the payload unrestricted. A future extension should allow events to carry a typed payload, so that asking whether an event fired and what value it carried can be expressed as a single check.

Request/response correlation. The formalism has no first-class way to express that a particular response is the answer to a particular request. A request/response pair can be approximated by combining a send event with a later receive event and tracking the in-flight requests in a `Set` entity, but matching a specific response back to the request it answers requires the request identifier to ride inside the event itself – which runs into the *Event payload* limitation above. Supporting request/response as a first-class pattern would let such requirements be expressed as a single relation rather than as a pair of separate predicates.

Required modifier keys. The current modifier system specifies which modifier keys are *permitted* for each entity type, but all modifiers are optional. A future extension could introduce a companion specification of which modifier keys are *required* for a given entity type. For example, predefined-event `Event` entities should always carry both a target reference and an event-type label.

Aggregate expression recognition. When a test case computes a result through a sequence of arithmetic operations (e.g. summing values and dividing by a count), the coverage checker must be able to recognise that the computation is equivalent to an aggregate such as average or median. Without this, a test exercising $avg(e,i)$ expressed via raw arithmetic would not be matched against a requirement that references the same aggregate by name. A future extension should define canonical aggregate expressions (*Avg*, *Median*, etc.) and a normalisation or equivalence procedure that maps common arithmetic patterns to them.

Triggered sequence construct. *Sequence* is existential: it requires at least one complete chain of ordered events to occur somewhere in the trace. The universal reading — every occurrence of a triggering event must be followed by the full sequence — cannot be expressed with the current constructs. A $TriggeredSequence(\psi_{trigger}, \psi_1, \ldots, \psi_n)$ construct would be needed, expanding to $\forall t \in T: \psi_{trigger}(\rho,t) \Rightarrow \exists t \leq t_1 \leq \cdots \leq t_n: \psi_i(\rho,t_i)$ for all i.

Dynamic entity lifetimes. The entity set E is fixed for the scope of a requirement, so the formalism cannot distinguish between an entity that does not exist at time t and one that merely has no value there – both reduce to $\sigma(e)$ being undefined. This limits the expression of requirements that involve entities with dynamic lifetimes, such as a spawned process or an opened connection.

Enum types. The base values set V_{base} currently contains only booleans, reals, time points, and intervals. Requirements that involve enumerated types (e.g. BACKUP/MASTER mastership modes, error codes, operating states) must represent enum values as untyped symbolic constants with no formal declaration or exhaustiveness guarantee. A future extension should introduce a named enum mechanism: a way to declare a finite set of symbolic constants (e.g. $Mastership = \{BACKUP, MASTER\}$) and extend V_{base} with their values, so that constraints can reference them by name and the coverage checker can enumerate their cases.

User-defined modifier keys. Currently, the set of permitted modifier keys for each entity type is fixed by M_E and can only be extended at the formalism level. A future extension could allow users to attach arbitrary key-value annotations to entities inline when writing a requirement – for example, to record a physical unit, an address, or a domain-specific tag – without requiring a change to M_E. The `Abstract` entity type already serves as a catch-all for entities that do not fit a predefined kind; user-defined modifiers would naturally complement it.

Global axioms. The formalism should define a set of predicates (axioms) that hold in every well-formed trace, ruling out physically impossible states. For example, two write events cannot be happening on the same register at the same time. Such axioms would constrain ρ and let the coverage checker reject inconsistent test cases.

3.5 Compatibility with well-known formal specification methods

Because the formalism is logic-agnostic, a requirement expressed in it is not tied to any single verification back-end. This subsection sketches how the temporal constructs correspond to established specification methods, so that a formalised requirement can be projected onto whichever back-end a downstream tool requires. The correspondences below are informal: they indicate the target operator each construct maps to, not a fully proven translation.

Linear Temporal Logic (LTL). The untimed constructs correspond directly to the standard LTL operators. A point-in-time predicate ψ plays the role of an LTL atomic proposition, and the temporal construct around it fixes the modality, as summarised in Table 2. Several of these are the classic specification patterns – *Causes* is the response pattern, *Precedes* the precedence pattern, and *Sequence* a chain of eventualities – which is what makes the mapping to LTL natural.

Metric and signal extensions (MTL, STL). Plain LTL captures ordering but not real-time bounds or arithmetic over values, both of which the formalism supports. Two well-known extensions of LTL close the gap. *Metric Temporal Logic* (MTL) adds time-bounded operators, which is what $CausesWithin(\psi_1, \psi_2, d)$ requires: it maps to $G(\psi_1 \rightarrow F_{[0,d]}\,\psi_2)$. *Signal Temporal Logic* (STL) additionally interprets atomic predi-

Table 2. Correspondence between the untimed temporal constructs and LTL. G, F, X, and W denote globally, eventually, next, and weak-until.

Construct	LTL equivalent
$\mathit{Always}(\psi)$	$G\,\psi$
$\mathit{Eventually}(\psi)$	$F\,\psi$
$\mathit{Initial}(\psi)$	ψ (initial state)
$\mathit{Immediately}(\psi_1, \psi_2)$	$G(\psi_1 \rightarrow X\,\psi_2)$
$\mathit{Causes}(\psi_1, \psi_2)$	$G(\psi_1 \rightarrow F\,\psi_2)$
$\mathit{Excludes}(\psi_1, \psi_2)$	$G\,\neg(\psi_1 \wedge \psi_2)$
$\mathit{Precedes}(\psi_1, \psi_2)$	$\neg\psi_2\ W\ \psi_1$
$\mathit{Sequence}(\psi_1, \ldots, \psi_n)$	$F(\psi_1 \wedge F(\psi_2 \wedge \cdots))$

cates as arithmetic comparisons over real-valued signals.

Timed automata. The same requirements also admit an operational reading as timed automata. A discrete-time requirement without timing bounds is a finite automaton over the entity valuations; the timed constructs introduce clocks: *CausesWithin* resets a clock when its condition holds and guards the effect by the bound d, and a sliding window is a clock measuring elapsed time since the window opened. This is the construction that timing-aware back-ends and model checkers consume.

Limits of the correspondence. Not every construct maps onto these methods without cost. The set quantifiers *ForAll* and *Exists*, together with *Filter* and *Size*, range over a finite set of entities and are therefore first-order; they exceed propositional LTL and must be unrolled over the (finite) entity set before an LTL or automaton encoding applies. *Sequence* and *Precedes* are expressible in LTL but expand into nested operators whose size grows with the number of stages. And *Causes*, with its unbounded eventuality, has no finite-clock timed-automaton equivalent: a liveness (acceptance) condition is required rather than a safety construction. These cases remain expressible, but the translation is no longer a one-to-one substitution.

4. Conclusions

This work presented a formal base for expressing software requirements and their test cases, designed to sit as an intermediate layer between natural language and the back-end logics used by verification tools. Unlike the surveyed approaches, which translate natural language directly into a single fixed target logic, the formalism is logic-agnostic and is shared between requirements and test cases, so that the same predicates appear on both sides – the property that makes mechanical detection of coverage gaps possible. Its constructs are named after the vocabulary used when describing requirements informally and are each tied to a precise mathematical definition, which keeps the representation readable to a human or an LLM while remaining unambiguous.

The formalism was defined in layers – primitives, entities, expressions and predicates, operations, temporal constructs, and finally requirements and test cases – and its design was justified against a set of properties drawn from real requirement examples. Its compatibility with well-known specification methods was sketched: the untimed temporal constructs correspond to LTL operators, the time-bounded and value-based constructs to its metric and signal extensions, and the same requirements admit an operational reading as timed automata. Known limitations and possible extensions were discussed openly, indicating where the present design trades expressiveness for simplicity and which directions a future revision should pursue.

Several avenues remain open. The most immediate is an empirical evaluation of how reliably an LLM translates natural-language requirements into the formalism, and of the coverage checker that consumes the result – both of which the present work defines the target for but does not yet measure. The limitations identified in Section 3.4 – among them parameterised modifiers, first-class predicates, typed and nested sets, and a request/response model – form a concrete agenda for extending the formalism as further requirement classes are encountered.

Acknowledgements

I would like to thank my supervisor Ing. Aleš Smrčka Ph.D. for his help.

References

[1] BEG, A., O'DONOGHUE, D. and MONAHAN, R. *A Short Survey on Formalising Software Requirements with Large Language Models*. 2025. Available at: https://arxiv.org/abs/2506.11874.

[2] COSLER, M., HAHN, C., MENDOZA, D., SCHMITT, F. and TRIPPEL, C. *Nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models*. 2023. Available at: https://arxiv.org/abs/2303.04864.

[3] FANG, W., LI, M., LI, M., YAN, Z., LIU, S. et al. AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs. In: *2024 IEEE LLM Aided Design Workshop (LAD)*. 2024, p. 1–1.

[4] Fazelnia, M., Mirakhorli, M. and Bagheri, H. Translation Titans, Reasoning Challenges: Satisfiability-Aided Language Models for Detecting Conflicting Requirements. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. New York, NY, USA: Association for Computing Machinery, 2024, p. 2294–2298.

[5] Ghosh, S., Elenius, D., Li, W., Lincoln, P., Shankar, N. et al. ARSENAL: Automatic Requirements Specification Extraction from Natural Language. In: Rayadurgam, S. and Tkachuk, O., ed. *NASA Formal Methods*. Cham: Springer International Publishing, 2016, p. 41–46.

[6] Giannakopoulou, D., Mavridou, A., Rhein, J., Pressburger, T., Schumann, J. et al. Formal Requirements Elicitation with FRET. In: *Joint Proceedings of REFSQ-2020 Workshops, Doctoral Symposium, Live Studies Track, and Poster Track*. Pisa, Italy: [b.n.], 2020.

[7] Jiang, A. Q., Li, W., Tworkowski, S., Czechowski, K., Odrzygóźdź, T. et al. Thor: Wielding Hammers to Integrate Language Models and Automated Theorem Provers. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2022, p. 8360–8373.

[8] Nayak, A., Timmapathini, H. P., Murali, V., Ponnalagu, K., Venkoparao, V. G. et al. Req2Spec: Transforming Software Requirements into Formal Specifications Using Natural Language Processing. In: *Requirements Engineering: Foundation for Software Quality (REFSQ 2022)*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 87–95.

[9] Nowakowski, W., Śmiałek, M., Ambroziewicz, A. and Straszak, T. Requirements-Level Language and Tools for Capturing Software System Essence. *Computer Science and Information Systems*. 2013, vol. 10, no. 4, p. 1499–1524.

[10] Quan, X., Valentino, M., Dennis, L. A. and Freitas, A. *Verification and Refinement of Natural Language Explanations through LLM-Symbolic Theorem Proving*. 2024. Available at: https://arxiv.org/abs/2405.01379.

[11] Yang, K., Swope, A., Gu, A., Chalamala, R., Song, P. et al. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023, p. 21573–21612.

Evaluating Test Completeness Against Software Requirements – Formalism

Bc. Jakub Bláha, Bc. Jan Klanica, Bc. Pavel Lukl, Bc. Timea Adamčíková

June 4, 2026

Contents

1 Introduction

- TODO: [Value transformations, like signed, etc.]

This document defines a logic-agnostic formal base for expressing software requirements and test cases. It is intended as a close-to-natural-language intermediate representation that an LLM (or a human) can read and produce: its constructs, expressions, and predicates are named after the vocabulary already used when describing requirements informally, while each name is tied to a precise mathematical definition. Its purpose is to enable mechanical detection of coverage gaps: given a set of requirements and a set of test cases formalised over a shared set of entities, a checker determines which requirement conditions are not exercised by any test.

The base is organised into several layers, each building on the ones below it. At the bottom, a time set and a set of *entities* (signals, storage, channels, states, events, and so on) describe the kinds of things present in the system, and a *trace* assigns to each time point a valuation mapping entities to their current values. Built on top of traces, *expressions* compute values and *predicates* evaluate to truth values at a given point in time. *Operations* are the actions a test case fires to drive a trace, each producing an effect predicate that states what must hold as a result of the firing. *Temporal constructs* then lift point-in-time predicates into statements about the whole trace by quantifying over time.

A requirement and a test case are both built from these same layers over the same entities, drawing their predicates from the same space; this shared vocabulary is what allows a checker to determine which conditions of a requirement a given test case exercises. The catalogues of entity kinds and operations are kept open to extension, so that supporting a new kind of entity or action requires only widening the relevant catalogue while every existing construct continues to work unchanged.

2 The primitives

These primitive sets serve as the base that the rest of the formalism builds on. The primitive sets are the following:

- T_D, T_C – the *discrete time* and *continuous time* sets respectively.
- ID_E – the set of entity identifiers.
- V_{base} – the base values set: booleans, reals, time points, and intervals.
- V_{nonset}, V – defined in Section 3 once entities are available; V_{nonset} is the union of base values and entities; V additionally includes finite sets whose elements are drawn from V_{nonset}.
- *EventTypeLabel* – the set of event type labels; defined below.

Definition 1 (Discrete time set)**.** *The* discrete time set T_D *is totally ordered and countable, extended with a greatest element* ∞*:*

$$T_D = \mathbb{N} \cup \{\infty\}$$

The ordering is the usual order on $\mathbb{N}$ *with* $\forall n \in \mathbb{N} : n < \infty$*.*

Definition 2 (Continuous time set)**.** *The* continuous time set T_C *is totally ordered and uncountable, extended with a greatest element* ∞*:*

$$T_C = \mathbb{R}_{\geq 0} \cup \{\infty\}$$

The ordering is the usual order on $\mathbb{R}_{\geq 0}$ *with* $\forall x \in \mathbb{R}_{\geq 0} : x < \infty$*.*

We say that T is an infinite totally ordered set; the time set can have *discrete* or *continuous* flavour; In the scope of a single requirement, T is equal either to T_D or T_C.

Definition 3 (Duration)**.** *A* duration *is a non-negative element of the time set, representing the length of a time span:*

$$\Delta = T$$

Thus $\Delta = \mathbb{N}$ *for discrete time and* $\Delta = \mathbb{R}_{\geq 0}$ *for continuous time.*

Definition 4 (Interval)**.** *An* interval *is a tuple of a start time point, a duration, and two endpoint-openness flags:*

$$\mathcal{I} = T \times \Delta \times \mathbb{B} \times \mathbb{B}$$

An interval $(t, d, \ell, r) \in \mathcal{I}$ *represents a time span with start* t*, end* $t + d$*, left-open flag* ℓ*, and right-open flag* r*:*

ℓ	r	*represented span*
false	*false*	$[t,\ t+d]$
true	*false*	$(t,\ t+d]$
false	*true*	$[t,\ t+d)$
true	*true*	$(t,\ t+d)$

Definition 5 (Entity identifier set)**.** *The* entity identifier set ID_E *is a finite nonempty set of unique abstract tokens called* entity identifiers*.*

Definition 6 (Base values)**.** *The* base values set V_{base} *is:*

$$V_{base} = \mathbb{B} \cup \mathbb{R} \cup T \cup \mathcal{I}$$

where $\mathbb{B} = \{\mathit{true}, \mathit{false}\}$ *is the set of booleans,* $\mathbb{R}$ *is the set of real numbers,* T *is the time set, and* $\mathcal{I}$ *is the set of intervals.*

Definition 7 (Event type label set)**.** *The* event type label set *EventTypeLabel is a finite set of abstract symbols:*

$$\begin{aligned} \mathit{EventTypeLabel} = \{\, &\mathit{generic},\ \mathit{calculated},\ \mathit{written},\ \mathit{read},\ \mathit{entered},\ \mathit{transmitted},\\ &\mathit{received},\ \mathit{inserted},\ \mathit{removed},\ \mathit{cleared}\,\} \end{aligned}$$

3 Entities and entity types

An entity type is an item belonging to a set of entity types. There is a number of defined entity types and this set is open to extension.

Definition 8 (Entity type set)**.**

$$T_E = \{\mathit{Signal}, \mathit{Storage}, \mathit{Event}, \mathit{EventTrigger}, \mathit{Channel}, \mathit{State}, \mathit{Value}, \mathit{Abstract}, \mathit{Set}\}$$

Entity type modifiers are properties which can be attached to an entity of a particular type. A modifier is either a *flag*, whose presence alone carries meaning, or a *parameterised modifier*, which additionally carries a value in *ModVal*.

Definition 9 (Modifier value set)**.** *The* modifier value set *is:*

$$\mathit{ModVal} = V_{base} \cup ID_E \cup \mathit{EventTypeLabel}$$

Each entity type is compatible with only a subset of the modifier keys. Therefore we define the *entity type modifiers function.*

Definition 10 (Entity type modifiers function)**.** $M_E : T_E \to \mathcal{P}(\mathit{ModKey})$ *maps each entity type to its set of permitted modifier keys, where ModKey is a finite nonempty set of modifier keys. The permitted modifier keys and their expected value types are:*

Entity type	***Modifier key***	***Value type***
Storage	*address*	$\mathbb{N}$
Event	*target*	ID_E
Event	*type*	*EventTypeLabel*

Every `Event` entity is expected to carry both modifiers: user-declared events set target to the identifier of their `EventTrigger` and type to generic; predefined events set target to their parent entity and type to the appropriate event type.

Note: At most one `Event` entity with a given $(target, type)$ pair should exist in E for a given requirement.

An entity is something that is referenced in a requirement and corresponding test cases. Formally, an entity is defined as a three-item tuple.

Definition 11 (Entity)**.**

$$e = (id_e, t_e, \mu_e)$$

, where

- id_e *is the unique identifier of the entity;* $id_e \in ID_E$
- t_e *is the type of the entity;* $t_e \in T_E$
- μ_e *is the modifier assignment function of the entity;* $\mu_e : ModKey \rightharpoonup ModVal$, $\mathrm{dom}(\mu_e) \subseteq M_E(t_e)$ [1]

No two entities in E share the same identifier: $\forall e, e' \in E : e \neq e' \Rightarrow id_e \neq id_{e'}$. We write E for the set of all entities. The set E is finite and nonempty.[2]

Definition 12 (Non-set values)**.** *The* non-set values V_{nonset} *is the union of base values and entities:*

$$V_{nonset} = V_{base} \cup E$$

Definition 13 (Values)**.** *The* values set V *is:*

$$V = V_{nonset} \cup \mathcal{P}_{fin}(V_{nonset})$$

where $\mathcal{P}_{fin}(V_{nonset})$ is the set of all finite subsets of V_{nonset}. Set values are flat: they may contain only non-set values.

4 Valuations and traces

Definition 14 (Valuation)**.** *A valuation is a partial function $\sigma : E \rightharpoonup V$ assigning a value to a subset of entities at each time point*[3]*; $\sigma(e)$ is undefined when the entity has no defined value at that time point. For entities of type `Event`, $\sigma(e)$ is always defined and $\sigma(e) \in \{true, false\}$, where $\sigma(e) = true$ iff the event occurs at that time point. For entities of type `Set`, $\sigma(e)$, when defined, is a finite subset of V_{nonset}, i.e. $\sigma(e) \in \mathcal{P}_{fin}(V_{nonset})$.*

We write Σ for the infinite set of all valuations.

Definition 15 (Trace)**.** *A trace is a function $\rho : T \to \Sigma$. We write $\rho(t)(e)$ for the value of entity e at time t.*

We write Σ^T for the set of all traces, i.e. the set of all functions from T to Σ.
[4]

5 Expressions and predicates

All expression functions defined in this section evaluate to some value $v \in V$. We write **Expr** for the infinite set of all expressions.

[1] $\mathrm{dom}(\mu_e)$ is a subset of the set of modifier keys compatible with the entity type t_e. It is not required that a value is defined for each modifier key compatible with t_e.

[2] The set E is finite in the scope of a single requirement.

[3] For instance a register entity can have its value undefined at time points preceeding a write to this register.

[4] This is standard function space notation: $\Sigma^T = \{\, \rho \mid \rho : T \to \Sigma \,\}$.

5.1 Expressions

5.1.1 Constants

Definition 16 (Constant)**.** *Any value $c \in V$ may appear as a constant expression.*

5.1.2 Arithmetic operators

Definition 17 (Arithmetic operators)**.** *The standard arithmetic operators $+$, $-$, $\times$, $/$ are defined on numeric values ($\mathbb{R}$ and its subsets) in the usual sense and can be used to construct expressions.*

5.1.3 Time functions

The following functions depend only on time points or intervals; they do not depend on a trace.

Definition 18 (Current time)**.** *$\mathit{Now} \in T$ returns the ambient time point t:*

$$\mathit{Now} \;=\; t$$

The following two functions are defined for **discrete time only** ($T = T_D = \mathbb{N}$).

Definition 19. *$\mathit{Prev} : T_D \rightharpoonup T_D$ returns the preceding time step:*

$$\mathit{Prev}(t) \;=\; t - 1$$

The function is partial: it is undefined at $t = 0$ since $t - 1 \notin T_D$.

Definition 20. *$\mathit{Next} : T_D \to T_D$ returns the following time step:*

$$\mathit{Next}(t) \;=\; t + 1$$

Definition 21. *$\mathit{RelTime} : T_D \times \mathbb{Z} \rightharpoonup T_D$ shifts a time point by n steps, where $n \in \mathbb{Z}$ may be positive or negative:*

$$\mathit{RelTime}(t,\, n) \;=\; t + n$$

The function is partial: it is undefined when $t + n < 0$ since $t + n \notin T_D$. Note that $\mathit{Prev}(t) = \mathit{RelTime}(t, -1)$ and $\mathit{Next}(t) = \mathit{RelTime}(t, 1)$.

The following functions are defined for both time flavours.

Definition 22. *$\mathit{Diff} : T \times T \to \Delta$ returns the duration between two time points:*

$$\mathit{Diff}(t_1,\, t_2) \;=\; |t_1 - t_2|$$

Definition 23 (Interval constructors)**.** *Four constructors build intervals from a start and end time point, differing only in endpoint openness. All are partial: undefined when $t_1 > t_2$.*

$$\begin{aligned}
\mathit{MkIntervalCC}(t_1,\, t_2) &\;=\; (t_1,\; t_2 - t_1,\; \mathit{false},\; \mathit{false}) && [t_1,\, t_2] \\
\mathit{MkIntervalOC}(t_1,\, t_2) &\;=\; (t_1,\; t_2 - t_1,\; \mathit{true},\; \mathit{false}) && (t_1,\, t_2] \\
\mathit{MkIntervalCO}(t_1,\, t_2) &\;=\; (t_1,\; t_2 - t_1,\; \mathit{false},\; \mathit{true}) && [t_1,\, t_2) \\
\mathit{MkIntervalOO}(t_1,\, t_2) &\;=\; (t_1,\; t_2 - t_1,\; \mathit{true},\; \mathit{true}) && (t_1,\, t_2)
\end{aligned}$$

Definition 24 (Since-zero interval)**.** *$\mathit{SinceZero} : T \to \mathcal{I}$ constructs an interval from time zero to a given time point t:*

$$\mathit{SinceZero}(t) \;=\; (0,\; t)$$

Definition 25 (Interval start)**.** *$\mathit{Start} : \mathcal{I} \to T$ returns the start time point of an interval:*

$$\mathit{Start}((t,\, d,\, \ell,\, r)) \;=\; t$$

Definition 26 (Interval end)**.** *$End : \mathcal{I} \to T$ returns the end time point of an interval:*

$$End((t,\, d,\, \ell,\, r)) \;=\; t + d$$

Definition 27 (Interval duration)**.** *$Duration : \mathcal{I} \to \Delta$ returns the duration of an interval:*

$$Duration((t,\, d,\, \ell,\, r)) \;=\; d$$

Definition 28 (Interval endpoint openness)**.** *$IsLeftOpen$, $IsRightOpen : \mathcal{I} \to \mathbb{B}$ return the endpoint-openness flags:*

$$IsLeftOpen((t,\, d,\, \ell,\, r)) \;=\; \ell \qquad IsRightOpen((t,\, d,\, \ell,\, r)) \;=\; r$$

Definition 29 (Time point interval membership)**.** *$InInterval : T \times \mathcal{I} \to \mathbb{B}$ holds when time point s falls within interval i:*

$$\begin{aligned} InInterval(s,\, i) \;\Leftrightarrow\; & \big(IsLeftOpen(i) \Rightarrow Start(i) < s\big) \\ \wedge\; & \big(\neg IsLeftOpen(i) \Rightarrow Start(i) \leq s\big) \\ \wedge\; & \big(IsRightOpen(i) \Rightarrow s < End(i)\big) \\ \wedge\; & \big(\neg IsRightOpen(i) \Rightarrow s \leq End(i)\big) \end{aligned}$$

5.1.4 Entity attribute accessors

These expressions return static properties of entities and do not depend on the trace or time.

Definition 30 (Address)**.** *For a `Storage` entity $e \in E$ carrying an address modifier, $Addr : E \to V$ returns the static address value:*

$$Addr(e) \;=\; \mu_e(address)$$

5.1.5 Trace functions

The following functions additionally take a trace $\rho \in \Sigma^T$ and produce values that reflect the state of the system over time.

Definition 31. *For a trace $\rho \in \Sigma^T$, the partial function $Val_\rho : E \times T \rightharpoonup V$ denotes the value of entity $e \in E$ at time $t \in T$:*

$$Val_\rho(e,\, t) \;=\; \rho(t)(e)$$

The function is partial: $Val_\rho(e, t)$ is undefined when $e \notin \mathrm{dom}(\rho(t))$.

Definition 32 (Value before time point)**.** *For a trace $\rho \in \Sigma^T$, the partial function $ValBefore_\rho : E \times T \rightharpoonup V$ is the left limit of the value of e at t:*

$$ValBefore_\rho(e,\, t) \;=\; \lim_{t' \to t^-} Val_\rho(e,\, t')$$

The function is partial: it is undefined when the left limit does not exist, i.e. when no operation has fired on e before t.

Definition 33 (Event occurrence count)**.** *For a trace $\rho \in \Sigma^T$ and an entity $ev \in E$ of type `Event`, $EvtOccCount_\rho : E \times \mathcal{I} \to \mathbb{N}$ returns the number of occurrences of ev within an interval $i \in \mathcal{I}$:*

$$EvtOccCount_\rho(ev,\, i) \;=\; \big|\{\, s \in T \mid InInterval(s,\, i) \;\wedge\; Val_\rho(ev,\, s) = true \,\}\big|$$

Definition 34 (Last occurrences function)**.** *For a trace $\rho \in \Sigma^T$, an entity $ev \in E$ of type `Event`, a time point $t \in T$, and $n \in \mathbb{N}^+$, let $t_1 \leq t_2 \leq \cdots \leq t_n$ be the n most recent occurrence times of ev up to t, i.e. the n largest elements of $\{\, s \in T \mid s \leq t \;\wedge\; Val_\rho(ev,\, s) = true \,\}$ sorted in ascending order. Then $LastOcc_\rho : E \times T \times \mathbb{N}^+ \rightharpoonup \mathcal{I}$ is defined as:*

$$LastOcc_\rho(ev,\, t,\, n) \;=\; MkIntervalCC(t_1,\, t_n)$$

The function is partial: it is undefined when fewer than n occurrences of ev exist up to t.

Note: The time of a single occurrence is obtained via $LastOcc_\rho(ev, t, 1)$, which returns an interval of duration 0. The occurrence time is then $Start(LastOcc_\rho(ev, t, 1))$.

Definition 35 (First occurrences function). *For a trace $\rho \in \Sigma^T$, an entity $ev \in E$ of type* `Event`*, a time point $t \in T$, and $n \in \mathbb{N}^+$, let $t_1 \leq t_2 \leq \cdots \leq t_n$ be the n earliest occurrence times of ev at or after t, i.e. the n smallest elements of $\{\, s \in T \mid s \geq t \;\wedge\; Val_\rho(ev, s) = true \,\}$ sorted in ascending order. Then $FirstOcc_\rho : E \times T \times \mathbb{N}^+ \rightharpoonup \mathcal{I}$ is defined as:*

$$FirstOcc_\rho(ev,\, t,\, n) \;=\; MkIntervalCC(t_1,\, t_n)$$

The function is partial: it is undefined when fewer than n occurrences of ev exist at or after t.

Note: The time of the first occurrence is $Start(FirstOcc_\rho(ev, t, 1))$.

Definition 36 (Maximum value function). *For a trace $\rho \in \Sigma^T$ and an entity $e \in E$ with an ordered value domain, $MaxVal_\rho : E \times \mathcal{I} \to V$ returns the maximum value of e over an interval $i \in \mathcal{I}$:*

$$MaxVal_\rho(e,\, i) \;=\; \sup\{\, Val_\rho(e,\, s) \mid s \in T,\ Start(i) \leq s \leq End(i) \,\}$$

Definition 37 (Minimum value function). *For a trace $\rho \in \Sigma^T$ and an entity $e \in E$ with an ordered value domain, $MinVal_\rho : E \times \mathcal{I} \to V$ returns the minimum value of e over an interval $i \in \mathcal{I}$:*

$$MinVal_\rho(e,\, i) \;=\; \inf\{\, Val_\rho(e,\, s) \mid s \in T,\ Start(i) \leq s \leq End(i) \,\}$$

5.1.6 Set functions

These functions operate on set values and do not depend on a trace.

Definition 38 (Set size). *$Size : \mathcal{P}_{fin}(V_{nonset}) \to \mathbb{N}$ returns the cardinality of a set:*

$$Size(s) \;=\; |s|$$

Definition 39 (Set filter). *For a set $s \in \mathcal{P}_{fin}(V_{nonset})$ and a predicate P parameterised by $x \in V_{nonset}$, $Filter : \mathcal{P}_{fin}(V_{nonset}) \times \mathbf{Pred} \to \mathcal{P}_{fin}(V_{nonset})$ returns the subset of elements satisfying P:*

$$Filter(s,\, P) \;=\; \{\, x \in s \mid P(x) \,\}$$

5.2 Predicates

A predicate is a function that evaluates to a value in $\mathbb{B}$. We write **Pred** for the infinite set of all predicates.

Definition 40 (Boolean constants). *$\top \in$ **Pred** and $\bot \in$ **Pred** are the always-true and always-false predicates respectively.*

Definition 41 (Comparison predicate). *For expressions $a, b \in V$ and a comparison operator $op \in \{=, \neq, <, \leq, >, \geq\}$:*

$$Cmp(a,\ op,\ b) \;=\; a\ op\ b$$

The operators $<, \leq, >, \geq$ require a, b to be from an ordered domain. Since $\Delta \subseteq \mathbb{R}$, durations are an ordered domain and duration comparison is a valid instance of Cmp.

Definition 42 (Set membership). *For a value $v \in V_{nonset}$ and a set value $s \in \mathcal{P}_{fin}(V_{nonset})$:*

$$Contains(v,\, s) \;\Leftrightarrow\; v \in s$$

Definition 43 (Negation). *For a predicate $P \in$ **Pred**:*

$$Not(P) \;=\; \neg P$$

Definition 44 (Conjunction and disjunction). *For a finite set of predicates $C \subseteq$ **Pred**:*

$$AllOf(C) \;=\; \bigwedge_{P \in C} P \qquad AnyOf(C) \;=\; \bigvee_{P \in C} P$$

Definition 45 (Implication)**.** *For predicates $A, B \in \mathbf{Pred}$:*

$$\mathit{Implies}(A,\, B) \;=\; A \Rightarrow B$$

where $\Rightarrow$ is the standard logical implication.

Definition 46 (Quantifiers)**.** *For a finite set S and a predicate $P(x)$ parameterised by $x \in S$:*

$$\mathit{ForAll}(S,\, P) \;=\; \bigwedge_{x \in S} P(x) \qquad \mathit{Exists}(S,\, P) \;=\; \bigvee_{x \in S} P(x)$$

Definition 47 (Event happening)**.** *For an entity $e \in E$ with $t_e = \mathit{Event}$ and a time point $t \in T$:*

$$\mathit{Happening}(e,\, t) \;\Leftrightarrow\; \mathit{Val}_\rho(e,\, t) = \mathit{true}$$

Definition 48 (Event has happened)**.** *For an entity $e \in E$ with $t_e = \mathit{Event}$ and a time point $t \in T$:*

$$\mathit{HasHappened}_\rho(e,\, t) \;\Leftrightarrow\; \exists t' \leq t :\; \mathit{Happening}(e,\, t')$$

The following predicates operate on intervals and durations. Since $\Delta \subseteq \mathbb{R}$, duration comparison is a special case of *Cmp* (Definition 41).

Definition 49 (Interval inclusion)**.** *Interval i_1 includes i_2 iff i_2 is entirely contained within i_1:*

$$\mathit{IntervalIncludes}(i_1,\, i_2) \;\Leftrightarrow\; \mathit{Start}(i_1) \leq \mathit{Start}(i_2) \;\wedge\; \mathit{End}(i_2) \leq \mathit{End}(i_1)$$

Definition 50 (Interval exclusion)**.** *Intervals i_1 and i_2 are exclusive iff they are completely disjoint:*

$$\mathit{IntervalExcludes}(i_1,\, i_2) \;\Leftrightarrow\; \mathit{End}(i_1) < \mathit{Start}(i_2) \;\vee\; \mathit{End}(i_2) < \mathit{Start}(i_1)$$

Definition 51 (Start included)**.** *The start of i_1 falls within i_2:*

$$\mathit{StartOfFirstIntervalIn}(i_1,\, i_2) \;\Leftrightarrow\; \mathit{Start}(i_2) \leq \mathit{Start}(i_1) \leq \mathit{End}(i_2)$$

Definition 52 (End included)**.** *The end of i_1 falls within i_2:*

$$\mathit{EndOfFirstIntervalIn}(i_1,\, i_2) \;\Leftrightarrow\; \mathit{Start}(i_2) \leq \mathit{End}(i_1) \leq \mathit{End}(i_2)$$

6 Operations

An operation is an action that the system performs at a specific point in time. Each operation has a set of arguments and produces *effect predicates* — a formal statement, parameterised by the firing time t, of what must hold in the trace as a result of the operation being performed. The operation is applied to some entity, which means that the argument list of every operation must include the entity the operation is applied to.

Definition 53 (Operation)**.** *An* operation *is a function*

$$op : T \times E \times A_1 \times \cdots \times A_n \to \mathbf{Pred}$$

for some $n \geq 0$ and additional argument types $A_i \subseteq V \cup E$, where T is the time set and E is the mandatory target entity. The predicate $op(t,\, e,\, a_1,\, \ldots,\, a_n)$ is the effect *of op applied to entity e at time t – it describes the condition that the operation causes to hold. The trigger condition that invokes an operation is specified at the requirement level. We write O for the finite set of all operations.*

Definition 54 (Entity type operations function)**.**

$$O_T : T_E \to \mathcal{P}(O)$$

maps each entity type and set of modifiers to its applicable operations.

The concrete mapping, including argument types and effects, is as follows. In effect predicates, t_{next} denotes the next time *any* value-modifying operation fires on entity e after t, or ∞ if no such firing occurs:

$$t_{next} = \begin{cases} \min\{\, t' \in T \mid t' > t \;\wedge\; \text{a value-modifying } op' \in O_T(t_e) \text{ fires on } e \text{ at } t' \,\} & \text{if such } t' \text{ exists} \\ \infty & \text{otherwise} \end{cases}$$

, where t is the time of the current firing and value-modifying operations are those whose effect predicate contains a $Val_\rho(e, -)$ term (i.e. *calculate*, *write*, *set_state*, *transmit*, *insert*, *remove*, *clear*, and the destination side of *read* and *receive*).

In the effect column, $EvtPred(e, ek, t)$ is used as shorthand for:

$$\forall\, ev \in E : \; \big(t_{ev} = \text{Event} \;\wedge\; \mu_{ev}(target) = id_e \;\wedge\; \mu_{ev}(type) = ek\big) \;\Rightarrow\; Val_\rho(ev, t) = true$$

i.e. every `Event` entity declared with the matching *target* and *type* modifiers has its value set to *true* at t, which by definition causes $Happening(ev, t)$ to hold.

Operation	**Notes**	**Effect predicates**
$calculate(t, e, expr)$	$t \in T$, $e \in E,\; t_e = \text{Signal}$, $expr \in \mathbf{Expr}$	$EvtPred(e, calculated, t)$ $\forall t' \in [t, t_{next}) : \; Val_\rho(e, t') = expr(\rho, t)$
$write(t, e, expr)$	$t \in T$, $e \in E,\; t_e = \text{Storage}$, $expr \in \mathbf{Expr}$	$EvtPred(e, written, t)$ $\forall t' \in [t, t_{next}) : \; Val_\rho(e, t') = expr(\rho, t)$
$read(t, e_{src}, e_{dst})$	$t \in T$, $e_{src} \in E,\; t_{e_{src}} = \text{Storage}$, $e_{dst} \in E,\; t_{e_{dst}} = \text{Signal}$	$EvtPred(e_{src}, read, t)$ $\forall t' \in [t, t_{next}) : \; Val_\rho(e_{dst}, t') = Val_\rho(e_{src}, t)$
$fire(t, e)$	$t \in T$, $e \in E,\; t_e = \text{EventTrigger}$	$EvtPred(e, generic, t)$
$set\_state(t, e, s)$	$t \in T$, $e \in E,\; t_e = \text{State}$, $s \in V$	$EvtPred(e, entered, t)$ $\forall t' \in [t, t_{next}) : \; Val_\rho(e, t') = s$
$transmit(t, e, expr)$	$t \in T$, $e \in E,\; t_e = \text{Channel}$, $expr \in \mathbf{Expr}$	$EvtPred(e, transmitted, t)$ $\forall t' \in [t, t_{next}) : \; Val_\rho(e, t') = expr(\rho, t)$
$receive(t, e_{src}, e_{dst})$	$t \in T$, $e_{src} \in E,\; t_{e_{src}} = \text{Channel}$, $e_{dst} \in E,\; t_{e_{dst}} = \text{Signal}$	$EvtPred(e_{src}, received, t)$ $Val_\rho(e_{dst}, t) = Val_\rho(e_{src}, t)$
$insert(t, e, v)$	$t \in T$, $e \in E,\; t_e = \text{Set}$, $v \in V_{\text{nonset}}$	$EvtPred(e, inserted, t)$ $\forall t' \in [t, t_{next}) : \; Val_\rho(e, t') = ValBefore_\rho(e, t) \cup \{v\}$
$remove(t, e, v)$	$t \in T$, $e \in E,\; t_e = \text{Set}$, $v \in V_{\text{nonset}}$	$EvtPred(e, removed, t)$ $\forall t' \in [t, t_{next}) : \; Val_\rho(e, t') = ValBefore_\rho(e, t) \setminus \{v\}$
$clear(t, e)$	$t \in T$, $e \in E,\; t_e = \text{Set}$	$EvtPred(e, cleared, t)$ $\forall t' \in [t, t_{next}) : \; Val_\rho(e, t') = \emptyset$

7 Temporal constructs

So far, predicates ($\psi \in \textbf{Pred}$) are point-in-time: they say whether something holds at a specific time t in a specific trace ρ. A *trace predicate* lifts this to the whole trace: it says whether something holds across the entire execution, not just at one moment. Formally, a trace predicate is a function $P : \Sigma^T \to \mathbb{B}$ — it takes a complete trace and returns true or false. A trace ρ *satisfies* a requirement R (written $\rho \models R$) iff R's constraint, expressed as a trace predicate P, gives $P(\rho) = \mathit{true}$.

The temporal constructs below build trace predicates from point-in-time predicates $\psi \in \textbf{Pred}$ by quantifying over time — expressing things like "ψ must hold at all times" or "whenever ψ_{cond} holds, ψ_{effect} must eventually follow."

Definition 55 (Trace predicate)**.** *A* trace predicate *is a function $P : \Sigma^T \to \mathbb{B}$. We write* **TracePred** *for the set of all trace predicates. A trace ρ* satisfies *a trace predicate P (written $\rho \models P$) iff $P(\rho) = \mathit{true}$.*

Definition 56 (Temporal constructs)**.**

$$\begin{aligned}
\mathit{Always}(\psi) &\Leftrightarrow \forall t \in T:\ \psi(\rho, t)\\
\mathit{Eventually}(\psi) &\Leftrightarrow \exists t \in T:\ \psi(\rho, t)\\
\mathit{Initial}(\psi) &\Leftrightarrow \psi(\rho, 0)\\
\mathit{Causes}(\psi_{cond},\, \psi_{effect}) &\Leftrightarrow \forall t \in T:\ \psi_{cond}(\rho, t) \Rightarrow \exists t' \geq t:\ \psi_{effect}(\rho, t')\\
\mathit{CausesWithin}(\psi_{cond},\, \psi_{effect},\, d) &\Leftrightarrow \forall t \in T:\ \psi_{cond}(\rho, t) \Rightarrow \exists t' \in [t,\, t+d]:\ \psi_{effect}(\rho, t')\\
\mathit{Sequence}(\psi_1,\, \ldots,\, \psi_n) &\Leftrightarrow \exists t_1 \leq t_2 \leq \cdots \leq t_n \in T:\ \psi_i(\rho,\, t_i)\ \mathit{for\ all}\ i\\
\mathit{Precedes}(\psi_1,\, \psi_2) &\Leftrightarrow \forall t_1 \in T:\ \psi_2(\rho,\, t_1) \Rightarrow \exists t_2 \leq t_1:\ \psi_1(\rho,\, t_2)\\
\mathit{Excludes}(\psi_1,\, \psi_2) &\Leftrightarrow \forall t \in T:\ \neg\big(\psi_1(\rho,\, t) \wedge \psi_2(\rho,\, t)\big)\\
\mathit{Immediately}(\psi_1,\, \psi_2) &\Leftrightarrow \forall t_1 \in T:\ \psi_1(\rho,\, t_1) \Rightarrow \psi_2(\rho,\, \mathit{Next}(t_1))
\end{aligned}$$

where ψ, ψ_{cond}, ψ_{effect}, $\psi_1, \ldots, \psi_n \in$ **Pred** *are point-in-time predicates and $d \in \Delta$ is a duration bound.*

The constructs compose via standard logical operators on trace predicates: conjunction $P_1 \wedge P_2$, disjunction $P_1 \vee P_2$, negation $\neg P$, implication $P_1 \Rightarrow P_2$ (if P_1 holds for trace ρ then P_2 must also hold), and biconditional $P_1 \Leftrightarrow P_2$. These operate on whole traces, not at individual time points.

8 Requirements & Test cases

Both requirements and test cases operate over the same shared entity set E, which is what makes coverage checking possible.

A requirement is a formal constraint on the system's behaviour, expressed as a trace predicate.

Definition 57 (Requirement)**.** *A* requirement *is a triple $R = (\mathit{flavour},\, \mathit{entities},\, \mathit{constraint})$ where:*

- *$\mathit{flavour} \in \{\mathit{Discrete},\, \mathit{Continuous}\}$ fixes the time set T for this requirement (T_D or T_C respectively)*
- *$\mathit{entities} \subseteq E$ is the set of entities participating in this requirement*
- *$\mathit{constraint} \in$* **TracePred** *is the formal constraint on the trace*

A trace ρ satisfies *R (written $\rho \models R$) iff $\mathit{constraint}(\rho) = \mathit{true}$.*

A test case exercises a subset of the same entities to produce a concrete trace, which the coverage checker then analyses against the requirement's constraint.

Definition 58 (Stimulus)**.** *A* stimulus *is a tuple $(t,\, op,\, e,\, a_1,\, \ldots,\, a_n)$ where:*

- $t \in T$ *is the firing time*
- *op is an operation name from the operations table*
- $e \in E$ *is the target entity*
- $a_1, \ldots, a_n \in V \cup E$ *are the additional arguments required by op*

We write **Stim** *for the set of all well-typed stimuli.*

Definition 59 (Test case)**.** *A* test case *is a triple* $\mathit{TestCase} = (\mathit{setup}, \mathit{stimuli}, \mathit{assertions})$ *where:*

- *setup is a valuation* $\sigma_0 : E \rightharpoonup V$ *establishing the initial state of the relevant entities*
- $\mathit{stimuli} \in \mathcal{P}_{\text{fin}}(\mathbf{Stim})$ *is a finite set of stimuli*
- $\mathit{assertions} : T \to \mathcal{P}(\mathbf{Pred})$ *maps each time point to the set of point-in-time predicates the test verifies at that time; it has finite support*